Multithreading – Seat Reservation System using Synchronization

Observation Task: Week-12 (Experiment No: 12, CO3) Date: __ / __ / 2026

■ Objective / Aim

To implement a seat reservation system in Java using multithreading, in which several person threads try to book seats and the synchronized reserve method allows only one person thread to access it at any time. The program also demonstrates creating multiple threads and setting thread priorities.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

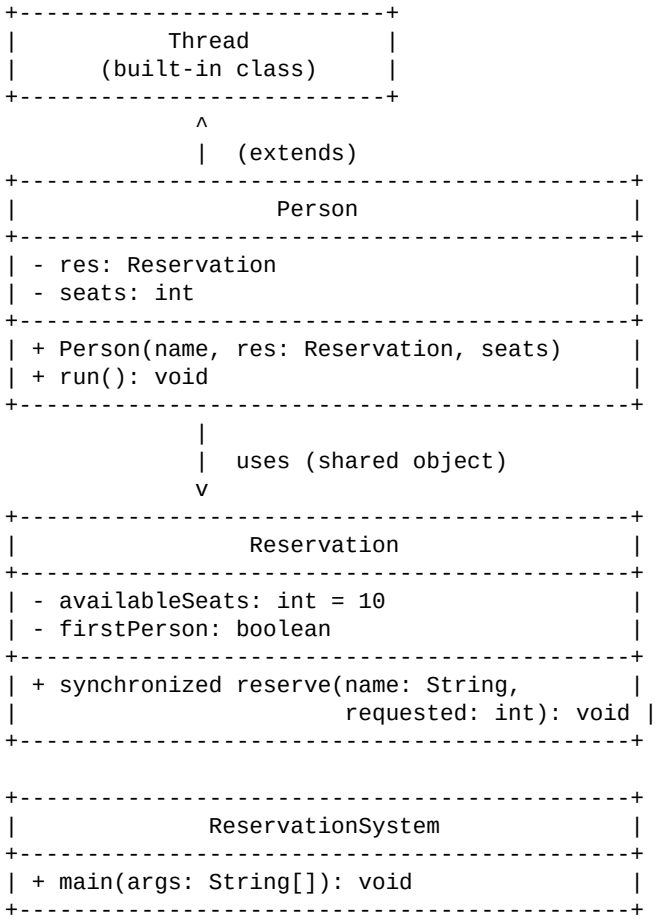
Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a class **Reservation** that holds the shared data: the number of available seats (initially 10).
2. Define the method **synchronized void reserve(String name, int requested)** in Reservation. The **synchronized** keyword locks the object, so only one thread can execute this method at a time.
3. Inside reserve():
   - Print that the person entered, then the available seats and the requested seats.
   - If requested seats <= available seats, print "Seat Available. Reserve now :-)", reduce the available seats and print the number of seats reserved.
   - Otherwise, print "Requested seats not available :-)".
   - Print that the person is leaving.
4. Create a class **Person** that extends Thread. Its constructor takes the person name, the shared Reservation object and the number of seats; its run() method calls reserve().
5. In main(), create one Reservation object and three Person threads (Person-1 requests 5 seats, Person-2 requests 2 seats, Person-3 requests 4 seats), all sharing that same object.
6. Set thread priorities with setPriority(): Person-1 = MAX_PRIORITY (10), Person-2 = NORM_PRIORITY (5), Person-3 = MIN_PRIORITY (1).
7. Start the threads one after another with a short delay (Thread.sleep) so that they request the lock in the order Person-1, Person-2, Person-3. While one person is inside reserve(), the next person has to wait.
8. Call join() on every thread so that main() waits until all persons have finished.

■ UML Class Diagram



Relationship: Person inherits from the built-in Thread class, so every Person object is a thread. ReservationSystem creates one Reservation object and three Person threads; all the threads share (association) that single Reservation object. Because reserve() is synchronized, the threads access it one at a time, which keeps the seat count consistent.

■ Source Code

```
// Shared resource: only one person thread can be inside reserve() at a time
class Reservation {
    private int availableSeats = 10;
    private boolean firstPerson = true;

    // synchronized => only one thread at a time (monitor lock on this object)
    synchronized void reserve(String name, int requested) {
        if (!firstPerson) {
            System.out.println("-------------------------------------------");
        }
        firstPerson = false;

        System.out.println(name + " entered.");
        System.out.println("Available seats: " + availableSeats
                + " Requested seats: " + requested);

        try {
            Thread.sleep(300);          // simulate time taken to book
        } catch (InterruptedException e) {
            System.out.println("Interrupted");
        }

        if (requested <= availableSeats) {
            System.out.println("Seat Available. Reserve now :-)");
            availableSeats -= requested;
```

```
            System.out.println(requested + " seats reserved.");
        } else {
            System.out.println("Requested seats not available :-)");
        }
        System.out.println(name + " leaving.");
    }
}

// Each person is a separate thread
class Person extends Thread {
    private Reservation res;
    private int seats;

    Person(String name, Reservation res, int seats) {
        super(name);
        this.res = res;
        this.seats = seats;
    }

    public void run() {
        res.reserve(getName(), seats);
    }
}

public class ReservationSystem {
    public static void main(String[] args) throws InterruptedException {
        Reservation res = new Reservation();

        Person p1 = new Person("Person-1", res, 5);
        Person p2 = new Person("Person-2", res, 2);
        Person p3 = new Person("Person-3", res, 4);

        // Thread priorities (1 = MIN, 5 = NORM, 10 = MAX)
        p1.setPriority(Thread.MAX_PRIORITY);
        p2.setPriority(Thread.NORM_PRIORITY);
        p3.setPriority(Thread.MIN_PRIORITY);

        p1.start();
        Thread.sleep(200);
        p2.start();
        Thread.sleep(200);
        p3.start();

        p1.join();
        p2.join();
        p3.join();
    }
}
```

■ Compilation & Execution Commands

Save the source code exactly as:

`ReservationSystem.java`

Compile the program:

```
javac ReservationSystem.java
```

Run the compiled program:

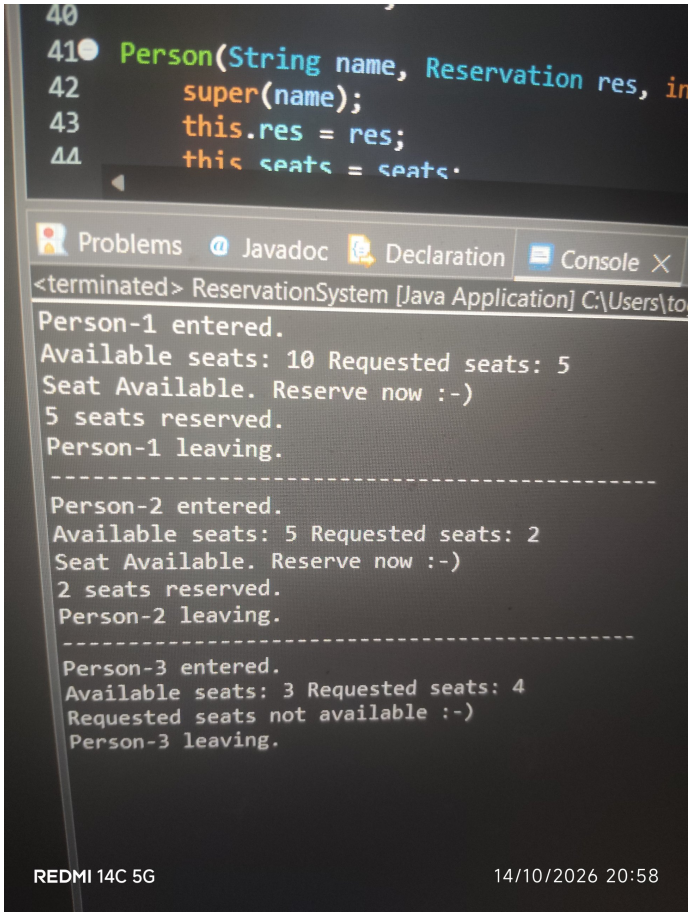
```
java ReservationSystem
```

■ Output Screen

Console output

```
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
-------------------------------------------
Person-2 entered.
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
-------------------------------------------
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
Person-3 leaving.
```

Screenshot – actual run in Eclipse



■ Result

The seat reservation program was successfully implemented in Java using multithreading. Three person threads (with different priorities) shared one Reservation object, and the synchronized reserve() method allowed only one person to book at a time. Person-1 (5 seats) and Person-2 (2 seats) were reserved successfully, while Person-3 (4 seats) was refused because only 3 seats were left.